Lecture Notes: TwinCAT 3

Data Types, Pointers, and Arrays

March 14, 2026

Contents

1 Introduction to Variables and Static Typing **2**
- 1.1 Variables as Containers . . . 2
- 1.2 Static vs. Dynamic Typing . . . 2
- 1.3 Declaration Block . . . 2

2 Standard Data Types **2**
- 2.1 Integer and Bit-Oriented Types . . . 2
  - 2.1.1 Integer Sizes . . . 2
  - 2.1.2 Base Notation . . . 2
  - 2.1.3 Bit Accessing . . . 2
- 2.2 Floating Point and Strings . . . 3
- 2.3 Time and Date . . . 3

3 User-Defined Types: Enumerations (ENUMs) **3**
- 3.1 Definition . . . 3
- 3.2 Key Points . . . 4

4 Memory Operations and SIZEOF **4**
- 4.1 Memory Allocation . . . 4
- 4.2 SIZEOF Operator . . . 4

5 Pointers vs. References **4**
- 5.1 Pointers . . . 4
  - 5.1.1 Operators . . . 5
  - 5.1.2 Risks . . . 5
- 5.2 References . . . 5
  - 5.2.1 Operators . . . 5
  - 5.2.2 Advantages . . . 6
  - 5.2.3 Safety Check . . . 6

6 Arrays **6**
- 6.1 Definition . . . 6
- 6.2 Key Features . . . 6
- 6.3 2D Array Declaration Comparison . . . 7

7 Type Conversion **7**
- 7.1 Implicit vs. Explicit Conversion . . . 7
- 7.2 Data Loss Warning . . . 7
- 7.3 Common Conversion Operators . . . 7

Quick Reference Card **8**

1 Introduction to Variables and Static Typing

1.1 Variables as Containers

Variables label and store data in memory so programs are human-readable and data can be manipulated.

1.2 Static vs. Dynamic Typing

Unlike JavaScript or Python (dynamic), **IEC 61131-3 is statically typed**:

Aspect	Static Typing (IEC 61131-3)	Dynamic Typing (JS/Python)
Type Checking	Compile time	Runtime
Type Declaration	Required before use	Not required
Type Changes	Not allowed during execution	Allowed

Table 1: Static vs. Dynamic Typing Comparison

1.3 Declaration Block

In TwinCAT 3, variables are declared between the keywords `VAR` and `END_VAR`.

Example: Basic Declaration

```
VAR
    fCabinetTemperature : REAL;  // Declares a 4-byte floating point variable
END_VAR
```

2 Standard Data Types

2.1 Integer and Bit-Oriented Types

2.1.1 Integer Sizes

Type	Size	Signed Range	Unsigned Type	Unsigned Range
SINT/USINT	1 byte	-128 to 127	USINT	0 to 255
INT/UINT	2 bytes	-32,768 to 32,767	UINT	0 to 65,535
DINT/UDINT	4 bytes	-2^{31} to $2^{31}-1$	UDINT	0 to $2^{32}-1$
LINT/ULINT	8 bytes	-2^{63} to $2^{63}-1$	ULINT	0 to $2^{64}-1$

Table 2: Integer Data Types

2.1.2 Base Notation

You can define integers using different bases:

2.1.3 Bit Accessing

Access individual bits using dot notation: `nVariable.0` (first bit)

Base	Prefix	Example
Binary	2#	2#1011
Octal	8#	8#17
Hexadecimal	16#	16#FF

Table 3: Base Notation Prefixes

Example: Integer Bases and Bit Access

```
VAR
    byBinaryValue   : BYTE := 2#11001010;    // Binary notation (decimal 202)
    wHexValue       : WORD := 16#FF0A;       // Hexadecimal notation (decimal 65290)
    nStatusFlags    : BYTE;
    bBitThree       : BOOL;
END_VAR

// Accessing the 4th bit (index 3) of nStatusFlags
bBitThree := nStatusFlags.3;
```

2.2 Floating Point and Strings

Type	Size	Equivalent	Description
REAL	4 bytes	Float	Single precision
LREAL	8 bytes	Double	Double precision
STRING	81 bytes (default)	—	ASCII-coded, 80 chars + null terminator
WSTRING	Variable	—	UNICODE/UTF-16, 2–4 bytes per character

Table 4: Floating Point and String Types

2.3 Time and Date

TwinCAT provides specific types to handle time in human-readable formats:

Type	Resolution	Prefix	Example
TIME	1ms	T#	T#2s, T#100ms
LTIME	1ns	LTIME#	LTIME#500us
DATE	1 day	D#	D#2024-01-15
TIME_OF_DAY	1ms	TOD#	TOD#14:30:00
DATE_AND_TIME	1ms	DT#	DT#2024-01-15-14:30:00

Table 5: Time and Date Types

3 User-Defined Types: Enumerations (ENUMs)

3.1 Definition

A set of named integral constants used to make code self-explanatory.

3.2 Key Points

- **Scope:** ENUMs are **globally accessible** in TwinCAT
- **Risk:** Can lead to "namespace pollution" if multiple libraries use the same ENUM names
- **Creation:** Created as a **DUT (Data Unit Type)** in the project tree

Example: DUT Definition

```
// DUT Definition (created as a separate DUT file in the project tree
    )
{attribute 'qualified_only'}
TYPE E_TrafficLight :
(
    Red    := 0,
    Yellow := 1,
    Green  := 2
);
END_TYPE
```

Example: Usage in Program

```
// Usage in a Program or Function Block
VAR
    eCurrentLight : E_TrafficLight;
END_VAR

// Assigning using the dot identifier (required with 'qualified_only'
     attribute)
eCurrentLight := E_TrafficLight.Green;
```

Tip: Use the `{attribute 'qualified_only'}` pragma to require the full enum name (e.g., `E_TrafficLight.Green`) and avoid namespace conflicts.

4 Memory Operations and SIZEOF

4.1 Memory Allocation

- Memory is generally allocated **upfront** at compile time
- Dynamic allocation is typically **discouraged** in PLC programming

4.2 SIZEOF Operator

Used to determine the memory size (in bytes) of a variable or type.

5 Pointers vs. References

5.1 Pointers

Pointers represent a memory address.

Target	Returns
`STRING` (default)	81 bytes
Reference	Size of the **referenced object**
Pointer	Size of the **pointer itself** (e.g., 8 bytes on 64-bit)

Table 6: SIZEOF Operator Behavior

5.1.1 Operators

Operator	Purpose	Example
`ADR()`	Get address of variable	`pVar := ADR(nValue)`
`^`	Dereference (access value)	`pVar^ := 100`

Table 7: Pointer Operators

5.1.2 Risks

⚠ **Warning:** Pointers are **not type-safe**. You can accidentally point a STRING pointer at an INT variable, resulting in "garbage" data.

Example: Pointer Usage

```
VAR
    nMyValue    : INT := 100;
    pnMyPointer : POINTER TO INT;
END_VAR

// Assign the address of nMyValue to the pointer
pnMyPointer := ADR(nMyValue);

// Dereference the pointer to update the value
pnMyPointer^ := 250;  // nMyValue is now 250
```

5.2 References

References act as an **alias** for an object.

5.2.1 Operators

Operator	Purpose
`REFERENCE TO`	Declare a reference
`REF=`	Assign a reference

Table 8: Reference Operators

Feature	Benefit
Type Safety	Compiler prevents assigning to wrong data type
Syntax	No special dereferencing (^) needed

Table 9: Reference Advantages

5.2.2 Advantages

5.2.3 Safety Check

Use `__ISVALIDREF()` to check if a reference points to a valid memory location before use.

```
IF __ISVALIDREF(refMyVariable) THEN
    // Safe to use the reference
END_IF
```

6 Arrays

6.1 Definition

A series of elements of the **same type** in **contiguous memory**.

6.2 Key Features

Feature	Description
Flexible Indices	Can start/end at any integer (e.g., `ARRAY [1..5]`, `ARRAY [-1..1]`)
Multi-Dimensional	Support for 2D, 3D, etc. arrays

Table 10: Array Features

Example: Array Declaration and Access

```
VAR
    // 1D Array: 5 integers with indices 0 through 4
    anValues : ARRAY [0..4] OF INT := [10, 20, 30, 40, 50];

    // 2D Array - Alternative #1: Explicit row and column indices
    anMatrix1 : ARRAY [0..2, 0..3] OF INT;  // 3 rows, 4 columns

    // 2D Array - Alternative #2: Array of arrays
    anMatrix2 : ARRAY [0..2] OF ARRAY [0..3] OF INT;  // 3 rows, 4
    columns
END_VAR

// Accessing elements
anValues[2] := 35;            // Set the 3rd element to 35
anMatrix1[1, 2] := 99;        // Set row 1, column 2 to 99
anMatrix2[1][2] := 99;        // Equivalent access for array of arrays
```

Style	Syntax	Access Pattern
Alternative #1	`ARRAY [0..2, 0..3] OF INT`	`arr[row, col]`
Alternative #2	`ARRAY [0..2] OF ARRAY [0..3] OF INT`	`arr[row][col]`

Table 11: 2D Array Declaration Styles

6.3 2D Array Declaration Comparison

7 Type Conversion

7.1 Implicit vs. Explicit Conversion

Direction	Type	Example	Operator Required
Smaller → Larger	Implicit	USINT → UINT	No
Larger → Smaller	Explicit	UINT → USINT	Yes (`UINT_TO_USINT`)

Table 12: Type Conversion Rules

7.2 Data Loss Warning

Warning: Converting a large value to a smaller type truncates the high bytes.

Example:

```
Decimal 259 = Binary 00000001 00000011 (2 bytes)
                             |
                             v
Truncated to USINT = Binary 00000011 = Decimal 3
```

7.3 Common Conversion Operators

From	To	Operator
INT	REAL	`INT_TO_REAL()`
REAL	INT	`REAL_TO_INT()`
UINT	USINT	`UINT_TO_USINT()`
STRING	INT	`STRING_TO_INT()`

Table 13: Common Type Conversion Operators

Quick Reference Card

Declaration Syntax

```
VAR
    <name> : <TYPE>;                                // Basic
    <name> : <TYPE> := <initial_value>;             // With initialization
    <name> : ARRAY [<start>..<end>] OF <TYPE>;      // Array
    <name> : POINTER TO <TYPE>;                     // Pointer
    <name> : REFERENCE TO <TYPE>;                   // Reference
END_VAR
```

Common Prefixes (Hungarian Notation)

Prefix	Type	Example
`b`	BOOL	`bIsRunning`
`n`	INT/UINT/etc.	`nCounter`
`f`	REAL/LREAL	`fTemperature`
`s`	STRING	`sMessage`
`a`	ARRAY	`anValues`
`p`	POINTER	`pnData`
`ref`	REFERENCE	`refMotor`
`e`	ENUM	`eState`

Table 14: Hungarian Notation Prefixes

Last Updated: March 14, 2026